

Implementazione Client BLE

Andrea Zorzi – BillyApp

Versione 1.0.0
25th November 2025

Contents

1 Scopo e perimetro

Questo documento descrive l'implementazione lato client del modello BLE definito in *Architettura BLE lato Client e Server*.

- piattaforme: iOS ed Android;
- strato condiviso: Kotlin Multiplatform Mobile (KMM);
- BLE e UI: implementate in modo nativo per ogni piattaforma.

1.1 Perimetro

Questo documento si concentra sull'implementazione del client BLE, escludendo la parte backend e le funzionalità di UI. Argomenti trattati:

- logica di advertising, scanning e lettura dei dati;
- gestione locale dell'identificativo B_{id} ;
- integrazione con KMM per le API condivise;
- gestione errori e permessi.

Sono esclusi:

- architettura ed implementazione lato backend;
- dettagli del modello crittografico e rotazione chiavi AES;

2 Panoramica architetturale

L'architettura del client BLE si basa su una separazione tra logica condivisa e implementazioni specifiche per piattaforma.

UI La UI è implementata in modo nativo per ogni piattaforma:

- schermate, view-model, state management;
- stato dell'applicazione ed interazioni utente.

BLE La logica BLE è implementata in modo nativo per ogni piattaforma, e si occupa di:

- advertising di B_{id} ;
- scanning e parsing dei pacchetti B_{id} ;
- gestione ruoli: peripheral e central;
- monitoraggio stato Bluetooth, permessi e background modes.

KMM Il modulo KMM gestisce lo strato condiviso tra le piattaforme, e si occupa di:

- contratti API;
- client HTTP;
- repository condivisi per la persistenza locale.

2.1 Principi di progettazione

L'architettura del client BLE segue i seguenti principi di progettazione:

- **Separation of Concerns:** separazione chiara tra logica BLE, UI e strato condiviso;
- **Modularità:** componenti indipendenti e riutilizzabili;
- **Testabilità:** progettazione per facilitare i test unitari e di integrazione;
- **Scalabilità:** architettura che consente future estensioni e modifiche;
- **Performance:** ottimizzazione per ridurre consumo energetico e latenza;
- **Sicurezza:** gestione sicura dei dati e delle comunicazioni BLE.

3 Matrice di comportamento BLE

La seguente matrice riassume i comportamenti di advertising e scanning in base ai vari stati dell'applicazione.

3.1 Regole di scelta

La scelta del canale per trasmettere i 16 byte di B_{id} dipende da:

- piattaforma (iOS o Android);
- stato dell'applicazione (foreground/background);
- configurazione Android di comportamento dell'app in background.

3.2 Matrice

La matrice seguente riassume il comportamento per ruolo, piattaforma e stato dell'applicazione:

Advertiser / Scanner	FG iOS	BG iOS	FG Android	BG Android
FG iOS	GAP	GAP	GAP	GAP
BG iOS	GATT	GATT	GATT	GATT
FG Android	GAP	GAP	GAP	GAP
BG Android	GAP/GATT	GAP/GATT	GAP/GATT	GAP/GATT

Legenda:

- **GAP:** advertising/scanning tramite pacchetti BLE standard;
- **GATT:** advertising/scanning tramite servizi e caratteristiche GATT.
- **GAP/GATT:** su **Android in background** la modalità è determinata dall'impostazione utente ρ (selezionata dalla UI) e quindi è GAP oppure GATT in modo deterministico.

In **GAP** i 16 byte di B_{id} sono trasmessi come parte del payload di advertising standard. In **GATT** i 16 byte di B_{id} sono trasmessi tramite una caratteristica di un servizio GATT personalizzato.

3.3 Implementazione

3.3.1 Preludio

La strategia di discovery si basa sull'emissione di pacchetti di advertising con capacità standard di 31 B. Per consentire il filtraggio univoco dei dispositivi, è necessario definire un identificativo di servizio.

Definizione (U_{srv}): Sia

$$U_{\text{srv}} = 0xB177_{16}$$

il Service UUID a 16 bit. *Uso:* Inserito nel pacchetto di advertising per permettere allo scanner di rilevare e filtrare univocamente i dispositivi dell'app.

Per compatibilità con le API native (Android/iOS) che richiedono il formato esteso, si definisce inoltre la base standard:

Definizione ($U_{\text{srv}}^{(128)}$): Sia

$$U_{\text{srv}}^{(128)} = 0000B177-0000-1000-8000-00805F9B34FB_{16}$$

l'UUID a 128 bit canonico associato a U_{srv} secondo Bluetooth Base UUID.

Uso (vincolante):

- in **advertising GAP** si inserisce U_{srv} come 16 bit per minimizzare i byte;
- nelle **API native** (scan filter, pubblicazione servizio GATT) si usa sempre e solo $U_{\text{srv}}^{(128)}$.

Il budget dei byte nel pacchetto è calcolato sommando le strutture dati *AD Structures* necessarie:

Componente	Struttura Interna	Spazio
Flags	1 B Len + 1 B Type + 1 B Val	3 B
Service UUID (U_{srv})	1 B Len + 1 B Type + 2 B UUID	4 B
MSD (Payload B_{id})	1 B Len + 1 B Type + 2 B Co.ID + 16 B Data	20 B
Totale Utilizzato		27 B
Spazio Residuo		4 B

Table 1: Bytes nel pacchetto di Advertising

3.3.2 GAP

In modalità GAP, i 16 B di B_{id} sono incapsulati nel campo *Manufacturer Specific Data* (MSD). Per validare la struttura del payload a livello di stack Bluetooth, è necessario un identificativo aziendale nell'header.

Definizione (C_{id}): Sia

$$C_{\text{id}} = 0x00B1_{16}$$

il Company Identifier (fittizio). *Uso:* Header obbligatorio (primi 2 byte) del blocco Manufacturer Specific Data; identifica il formato proprietario dei dati successivi. *Codifica (vincolante):* nel blob MSD i 2 byte di C_{id} sono in **little-endian**, quindi $0x00B1$ è codificato come bytes $B1\ 00$.

La struttura risultante del blocco MSD è la seguente:

Campo	Dimensione	Descrizione
Length	1 B	Lunghezza dei dati successivi ($0 \times 13 = 19$ dec)
AD Type	1 B	Tipo di dato: $0 \times FF$ (Manufacturer Specific)
Company ID	2 B	Valore fisso C_{id}
Data Payload	16 B	L'identificativo B_{id} crittografato

Table 2: Struttura del blocco MSD

3.3.3 GATT (Fallback Strategy)

Poiché il sistema operativo iOS in background rimuove il campo MSD dai pacchetti di advertising, si implementa una strategia ibrida. Se lo scanner rileva U_{srv} ma il payload MSD è assente, deve stabilire una connessione GATT per recuperare il dato.

Definizione (U_{char}): Sia

$$U_{char} = B1770001-5E77-4C00-9000-ABCDEF123456_{16}$$

il Characteristic UUID a 128-bit. U_{so} : Endpoint di lettura esposto dal server GATT per il recupero del payload B_{id} in modalità connessa.

Il diagramma seguente illustra la logica decisionale dello scanner:

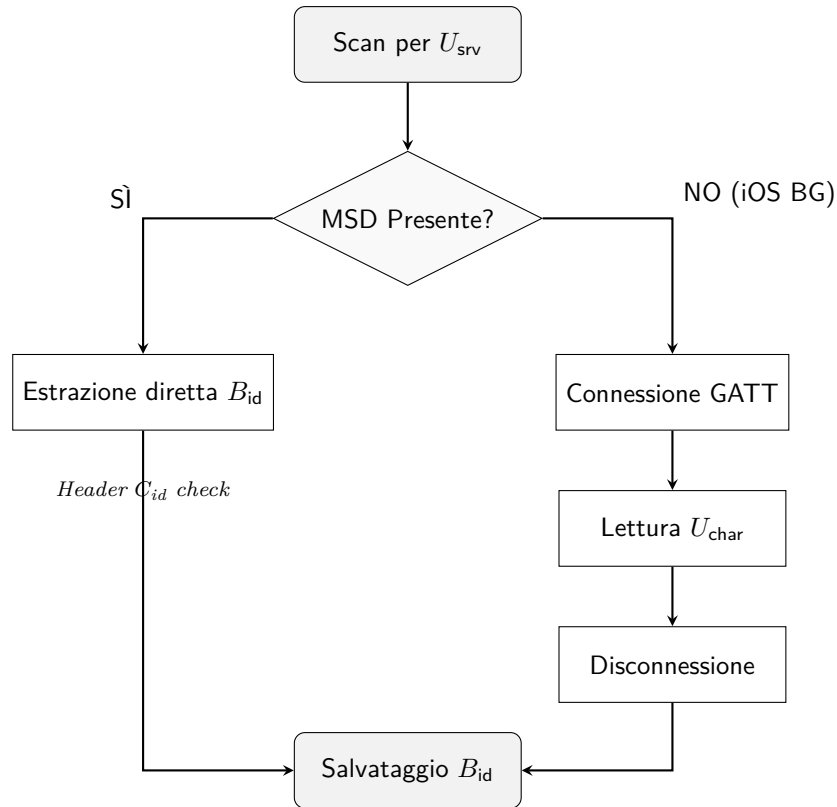


Figure 1: Flusso logico di acquisizione dati

4 Architettura dei Moduli Software

Il sistema è strutturato secondo il pattern *Native-Controlled*. In questo modello architetturale, i moduli nativi (iOS e Android) possiedono il controllo del flusso di esecuzione e delle risorse

hardware, mentre il modulo condiviso (KMM) funge da nucleo logico passivo.

Di seguito sono definiti i moduli principali e le loro responsabilità.

4.1 Strato Condiviso (KMM)

Il modulo `:shared` centralizza la logica di dominio, la persistenza dei dati grezzi e la gestione dello stato applicativo, garantendo la totale uniformità comportamentale tra le piattaforme.

Definizione (Core): Sia `Core` il componente di facciata (*Facade*) con solo stato volatile (cache in RAM), nessuno stato persistente che funge da unico punto di ingresso per i driver nativi. *Uso:* Gestione del protocollo di trasporto. Le sue responsabilità sono:

- validare formalmente i pacchetti BLE ricevuti e applicare la logica di deduplica volatile (Cache);
- calcolare e fornire il payload B_{id} corrente basandosi sulla sincronizzazione temporale e sul batch locale.

Definizione (SecureRepository): Sia `SecureRepository` il componente responsabile della persistenza sicura dei dati grezzi. *Uso:* Gestione del database locale cifrato (I/O su disco). Le sue responsabilità sono:

- memorizzare il materiale crittografico per l'advertising (batch dei payload);
- mantenere la coda FIFO persistente dei pacchetti rilevati in attesa di invio al backend.

Definizione (ResolvedRepository): Sia `ResolvedRepository` il componente che gestisce lo stato applicativo di alto livello. *Uso:* Sorgente di verità per l'interfaccia utente (UI). Le sue responsabilità sono:

- aggregare le risposte del backend e mantenere l'insieme dinamico degli utenti risolti ($\mathcal{A}$);
- applicare la logica di decadimento temporale (Time-To-Live) per rimuovere dalla vista gli utenti non più rilevati.

4.2 Modulo Android (:androidApp)

Il modulo Android è progettato per garantire l'esecuzione continua in background tramite i servizi di sistema.

Definizione (BleService): Sia `BleService` un componente architetturale di tipo *Foreground Service*. *Uso:* È il componente che sopravvive alla chiusura dell'interfaccia grafica. Le sue responsabilità sono:

- mantenere attivo il processo dell'applicazione;
- gestire le istanze dei driver `BluetoothLeAdvertiser` e `BluetoothLeScanner`;
- coordinare le chiamate verso `Core` in base agli eventi di sistema (timer e callback BLE).

Definizione (GattClientManager): Sia `GattClientManager` un componente ausiliario per la gestione delle connessioni. *Uso:* Gestisce le connessioni GATT puntuali. Le sue responsabilità sono:

- effettuare la connessione verso dispositivi che non espongono dati in modalità GAP (es. iOS Background);
- leggere la caratteristica specifica ed estrarne il payload.

4.3 Modulo iOS (:iosApp)

Il modulo iOS è strutturato per conformarsi al ciclo di vita rigoroso del framework CoreBluetooth.

Definizione (BleController): Sia BleController il componente singleton che gestisce lo stack Bluetooth. *Uso:* Agisce da *Delegate* per gli eventi di sistema. Le sue responsabilità sono:

- configurare e gestire il CBCentralManager (scansione) e il CBPeripheralManager (trasmissione);
- implementare la logica di *State Restoration* per garantire il riavvio silenzioso dell'app in background da parte del sistema operativo;
- orchestrare la strategia di discovery ibrida (GAP/GATT) interfacciandosi con Core.

5 Strato Condiviso (KMM)

Il modulo :shared è sviluppato in Kotlin puro (Kotlin/Common). Agisce come un sistema deterministico privo di dipendenze dirette dal sistema operativo: a parità di stessi input, stesso tempo logico e stesso stato persistente, produce output identici e trasformazioni di stato identiche.

Definizione (B_{id}): Sia

$$B_{id} \in \{0, 1\}^{128}$$

l'identificativo BLE cifrato trasmesso e ricevuto via radio. Definito in *Architettura BLE lato Client e Server*.

5.1 Core

Il componente Core implementa la logica di business pura e funge da orchestratore deterministico tra (i) input dei driver nativi e (ii) stato locale persistente gestito dai repository. Le interazioni con il backend avvengono tramite chiamate sincrone/asincrone effettuate dal client HTTP condiviso (KMM), ma Core resta privo di dipendenze dal ciclo di vita del sistema operativo: scheduling, timer e politiche di retry/backoff sono responsabilità del layer nativo.

5.1.1 Modello Temporale

Questa logica governa la selezione del payload da trasmettere basandosi sul tempo UTC.

Definizione (t_{unix}): Sia

$$t_{unix} \in \mathbb{N}, \quad [t_{unix}] = s,$$

il tempo corrente in secondi trascorsi dal 1 gennaio 1970. Definito in *Architettura BLE lato Client e Server*.

Definizione (Δt_{slot}): Sia

$$\Delta t_{slot} = 600 \text{ s} = 10 \text{ min}$$

la durata di uno slot temporale. Definito in *Architettura BLE lato Client e Server*.

Definizione (S_{slot}): Sia

$$S_{slot} = \left\lfloor \frac{t_{unix}}{\Delta t_{slot}} \right\rfloor \in \mathbb{N}$$

l'indice dello slot temporale corrente. Definito in *Architettura BLE lato Client e Server*.

Procedura (getCurrentBid): Sia

$$\text{getCurrentBid} : \emptyset \rightarrow \{0, 1\}^{128} \cup \{\text{null}\}$$

la funzione invocata dai timer nativi per ottenere il payload da advertising nello slot corrente.

Algoritmo:

1. **Calcolo Slot:** Si calcola S_{slot} a partire da t_{unix} e Δt_{slot} .
2. **Query Repository:** Si interroga la relazione $\mathcal{T}_{\text{batch}}$ (definita in *SecureRepository*). Si definisce

$$R = \{ b \mid (s, b) \in \mathcal{T}_{\text{batch}} \wedge s = S_{\text{slot}} \}.$$

Per vincolo di chiave primaria su s (vedi *SecureRepository*) vale $|R| \leq 1$.

3. **Restituzione:**

$$\text{Return } \begin{cases} b & \text{se } R = \{b\} \\ \text{null} & \text{se } R = \emptyset \end{cases}$$

5.1.2 Memoria Volatile

Questa logica governa l'ingestione dei pacchetti osservati, applicando un rate-limiting in RAM per proteggere l'I/O del dispositivo ed evitare accodamenti ridondanti. Nel modello si assume che i driver nativi validino il formato e forniscano in input sempre un B_{id} in $\{0, 1\}^{128}$.

Definizione (b): Sia

$$b \in \{0, 1\}^{128}$$

un payload ricevuto dai driver nativi, cioè un'istanza di B_{id} .

Definizione (τ): Sia

$$\tau = \frac{\Delta t_{\text{slot}}}{6}$$

la finestra minima tra due inserimenti dello stesso payload in coda.

Definizione ($\mathcal{C}$): Sia

$$\mathcal{C} \subset \{0, 1\}^{128} \times \mathbb{N}$$

la cache volatile, che associa a ciascun payload b l'ultimo timestamp t di ingestione.

Predicato (hit):

$$\text{hit}(b) \iff \exists (b, t_{\text{ins}}) \in \mathcal{C} \mid (t_{\text{unix}} - t_{\text{ins}}) < \tau.$$

la funzione che verifica la presenza recente di un payload, definita da:

Procedura (ingestPacket): Sia

$$\text{ingestPacket} : \{0, 1\}^{128} \rightarrow \emptyset$$

la funzione di ingestione dei pacchetti rilevati.

Algoritmo:

1. **Check Cache:** Se $\text{hit}(b)$ è vera, termina.
2. **Commit:** Il payload viene accodato in $\mathcal{T}_{\text{queue}}$ (definita in *SecureRepository*) tramite la procedura persistente *SecureRepository.enqueue*. La semantica è:

$$\exists k_{\text{new}} \in \mathbb{N} \text{ tale che } k_{\text{new}} > k \quad \forall (k, \tilde{b}) \in \mathcal{T}_{\text{queue}},$$

e si esegue

$$\mathcal{T}_{\text{queue}} \leftarrow \mathcal{T}_{\text{queue}} \cup \{(k_{\text{new}}, b)\}.$$

In implementazione, k_{new} è assegnato dal DB tramite chiave autoincrementale.

3. **Aggiornamento Cache:**

$$\mathcal{C} \leftarrow (\mathcal{C} \setminus \{(b, t) \mid (b, t) \in \mathcal{C}\}) \cup \{(b, t_{\text{unix}})\}.$$

5.1.3 Sincronizzazione Batch

Questa logica governa quando e come aggiornare il batch locale usato dall'advertising, garantendo che la funzione `getCurrentBid` sia definita (non-null) per lo slot corrente in condizioni di connettività.

Definizione (T_{batch}): Sia

$$T_{\text{batch}} = 24 \text{ h}$$

la durata di validità di un batch di B_{id} inviato dal backend al client. Definito in *Architettura BLE lato Client e Server*.

Definizione (N_{batch}): Sia

$$N_{\text{batch}} = \frac{T_{\text{batch}}}{\Delta t_{\text{slot}}}$$

il numero di B_{id} contenuti in un batch. Definito in *Architettura BLE lato Client e Server*.

Definizione ($\mathbf{B}$): Sia

$$\mathbf{B} = (B_{\text{id},j})_{j=1}^{N_{\text{batch}}}, \quad B_{\text{id},j} \in \{0, 1\}^{128},$$

una sequenza ordinata di payload ricevuta dal backend. Definito in *Architettura BLE lato Client e Server*.

Definizione (S_1): Sia

$$S_1 \in \mathbb{N}$$

l'indice del primo slot coperto dal batch $\mathbf{B}$. Definito in *Architettura BLE lato Client e Server*.

Procedura (`ensureAdvertisingBatch`): Sia

$$\text{ensureAdvertisingBatch} : \emptyset \rightarrow \emptyset$$

la procedura invocata periodicamente o su eventi (primo avvio, login, `getCurrentBid=null`) per garantire copertura del batch locale.

Algoritmo:

1. **Calcolo Slot:** Si calcola S_{slot} .
2. **Controllo Copertura:** Si definisce

$$S_{\text{max}} = \max\left(\{s \in \mathbb{N} \mid \exists b : (s, b) \in \mathcal{T}_{\text{batch}}\}\right)$$

se $\mathcal{T}_{\text{batch}} \neq \emptyset$. La copertura è considerata *insufficiente* se vale almeno una delle seguenti condizioni:

- (a) $\mathcal{T}_{\text{batch}} = \emptyset$;
 - (b) $\nexists b : (S_{\text{slot}}, b) \in \mathcal{T}_{\text{batch}}$;
 - (c) $\mathcal{T}_{\text{batch}} \neq \emptyset$ e $S_{\text{max}} - S_{\text{slot}} < \frac{N_{\text{batch}}}{6}$.
3. **Fetch dal backend (solo se insufficiente):** Se la copertura è insufficiente, si richiede al backend un nuovo batch $(S_1, \mathbf{B})$ tale che lo slot corrente sia coperto:

$$S_1 \leq S_{\text{slot}} \leq S_1 + N_{\text{batch}} - 1.$$

In caso di errore o assenza di connettività, lo stato locale non viene modificato.

4. **Commit Atomico:** Se il fetch ha successo, si invoca `SecureRepository.replaceBatch( $S_1, \mathbf{B}$ )`, che sostituisce $\mathcal{T}_{\text{batch}}$ in un'unica transazione (vedi *SecureRepository*).

5.1.4 Sincronizzazione Coda

Questa logica governa l'invio dei payload osservati al backend e l'aggiornamento dello stato UI in modo idempotente e consistente rispetto alla FIFO persistente, con una semantica 1:1 tra chiamata HTTP e payload processato.

Procedura (syncQueue): Sia

$$\text{syncQueue} : \emptyset \rightarrow \emptyset$$

la procedura invocata periodicamente dai layer nativi.

Algoritmo:

1. **Selezione Testa FIFO:** Si invoca `SecureRepository.peekHead()`. Se restituisce null, si invoca comunque

$$\text{ResolvedRepository.pruneActiveSet()}.$$

e termina. Altrimenti si ottiene (k_0, b_0) come testa FIFO da processare.

2. **Chiamata Backend (singolo payload):** Si invia al backend il solo payload b_0 per ottenere, al più, un singolo match applicativo.

La risposta viene modellata come un valore opzionale

$$R_{\text{srv}} \in \mathcal{M} \cup \{\perp\}.$$

dove $\mathcal{M}$ è definito in *ResolvedRepository* (match positivo per un utente) e $\perp$ rappresenta “nessun match” (payload non risolvibile o privo di utente associato).

In caso di errore di rete, timeout o risposta non valida, la procedura termina senza modificare lo stato persistente:

$$\mathcal{T}_{\text{queue}} \text{ resta invariata.}$$

3. **Ack e Dequeue Atomico:** Se la chiamata ha avuto esito tecnico positivo, si invoca in transazione

$$\text{SecureRepository.deleteByKey}(k_0).$$

4. **Aggiornamento Stato UI:** Se $R_{\text{srv}} = m$, si invoca

$$\text{ResolvedRepository.onMatchFound}(m).$$

Indipendentemente dal fatto che vi sia stato match o meno, al termine si invoca

$$\text{ResolvedRepository.pruneActiveSet}()$$

per applicare il decadimento temporale e mantenere $\mathcal{A}$ coerente rispetto a t_{unix} .

5.2 SecureRepository

Il repository gestisce lo stato persistente dei dati grezzi (*Raw Data*) e costituisce l'unico componente autorizzato a effettuare I/O su disco. Espone due strutture persistenti: (i) batch per l'advertising e (ii) coda FIFO di ingestione. Ogni modifica su disco avviene in transazione ACID (SQLite), in modo che gli invarianti dichiarati siano preservati anche in caso di crash.

5.2.1 Implementazione DB

La persistenza è implementata tramite un database SQLite cifrato *at-rest*.

- Il driver DB è fornito al KMM dal layer nativo (Android/iOS).
- La chiave di cifratura del DB è generata e conservata in un contenitore sicuro del sistema operativo (Android Keystore / iOS Keychain) e non transita mai in log né in storage non protetti.
- Le tabelle sono indicizzate sulle rispettive chiavi primarie per garantire:
 - lookup $O(1)$ sul batch (per s);
 - lettura testa FIFO tramite indice su k : ORDER BY k ASC LIMIT 1

5.2.2 Advertising Batch

Struttura *read-optimized* per il recupero $O(1)$ del payload da trasmettere nello slot corrente.

Definizione (s): Sia

$$s \in \mathbb{N}$$

un indice di slot temporale. Definito in *Architettura BLE lato Client e Server* tramite la discretizzazione S_{slot} .

Definizione ($\mathcal{T}_{\text{batch}}$): Sia

$$\mathcal{T}_{\text{batch}} \subset \mathbb{N} \times \{0, 1\}^{128}$$

la relazione persistente che associa a ciascun indice di slot s il payload b da trasmettere in quello slot.

Vincolo (chiave primaria su s):

$$\forall (s, b), (s, b') \in \mathcal{T}_{\text{batch}} \Rightarrow b = b'.$$

Vincolo (cardinalità massima):

$$|\mathcal{T}_{\text{batch}}| \leq N_{\text{batch}}.$$

Uso: garantisce che il batch locale non cresca senza bound; in condizioni normali contiene esattamente N_{batch} righe.

Procedura (replaceBatch): Sia

$$\text{replaceBatch} : \mathbb{N} \times \left(\{0, 1\}^{128}\right)^{N_{\text{batch}}} \rightarrow \emptyset$$

la procedura che sostituisce atomicamente $\mathcal{T}_{\text{batch}}$ con un nuovo batch $(S_1, \mathbf{B})$.

Algoritmo (transazione unica):

1. Si esegue $\mathcal{T}_{\text{batch}} \leftarrow \emptyset$.
2. Per ogni $j \in \{1, \dots, N_{\text{batch}}\}$ si inserisce:

$$\mathcal{T}_{\text{batch}} \leftarrow \mathcal{T}_{\text{batch}} \cup \{(S_1 + (j - 1), B_{\text{id}, j})\}.$$

5.2.3 Ingestion Queue

Struttura *write-optimized* per l'accumulo persistente dei payload osservati in attesa di invio al backend. L'ordine FIFO è indotto dalla chiave sequenziale crescente.

Definizione (k): Sia

$$k \in \mathbb{N}$$

un indice sequenziale persistente, monotono crescente, utilizzato come chiave primaria della coda.

Definizione ($\mathcal{T}_{\text{queue}}$): Sia

$$\mathcal{T}_{\text{queue}} \subset \mathbb{N} \times \{0, 1\}^{128}$$

la relazione persistente che modella la FIFO.

Vincolo (chiave primaria su k):

$$\forall (k, b), (k, b') \in \mathcal{T}_{\text{queue}} \Rightarrow b = b'.$$

Proprietà (FIFO indotta da k): Qualsiasi operazione di lettura/esportazione deve considerare gli elementi in ordine crescente di k , a partire dal minimo k presente.

Definizione (Q_{max}): Sia

$$Q_{\text{max}} = 50000$$

la capacità massima della coda persistente. *Uso:* limite di sicurezza per prevenire crescita illimitata in condizioni offline prolungate.

Vincolo (capacità):

$$|\mathcal{T}_{\text{queue}}| \leq Q_{\text{max}}.$$

Procedura (enqueue): Sia

$$\text{enqueue} : \{0, 1\}^{128} \rightarrow \emptyset$$

la procedura di inserimento persistente.

Algoritmo (transazione unica):

1. **Enforce Cap:** Se $|\mathcal{T}_{\text{queue}}| = Q_{\text{max}}$, si elimina il più vecchio elemento:

$$k_{\min} = \min\{k \in \mathbb{N} \mid \exists b : (k, b) \in \mathcal{T}_{\text{queue}}\},$$

e, per vincolo di chiave primaria su k , esiste un unico $b_{\min}$ tale che $(k_{\min}, b_{\min}) \in \mathcal{T}_{\text{queue}}$. Si esegue quindi:

$$\mathcal{T}_{\text{queue}} \leftarrow \mathcal{T}_{\text{queue}} \setminus \{(k_{\min}, b_{\min})\}.$$

2. **Insert:** Si inserisce un nuovo elemento con chiave crescente assegnata dal DB:

$$\exists k_{\text{new}} \in \mathbb{N} \text{ con } k_{\text{new}} > k \forall (k, \tilde{b}) \in \mathcal{T}_{\text{queue}},$$

$$\mathcal{T}_{\text{queue}} \leftarrow \mathcal{T}_{\text{queue}} \cup \{(k_{\text{new}}, b)\}.$$

Procedura (peekHead): Sia

$$\text{peekHead} : \emptyset \rightarrow (\mathbb{N} \times \{0, 1\}^{128}) \cup \{\text{null}\}$$

la procedura che restituisce la testa FIFO senza modificarla. *Uso:* query indicizzata ORDER BY k ASC LIMIT 1.

Procedura (deleteByKey): Sia

$$\text{deleteByKey} : \mathbb{N} \rightarrow \emptyset$$

la procedura che elimina esattamente l'elemento con chiave primaria k , in transazione. *Uso:* DELETE FROM queue WHERE k = :k.

5.3 ResolvedRepository

Il repository gestisce lo stato applicativo di alto livello mostrato in UI: l'insieme degli utenti risolti dal backend e la loro validità temporale. Non mantiene alcun mapping persistente $B_{id} \rightarrow$ utente, in accordo con *Architettura BLE lato Client e Server*. Lo stato $\mathcal{A}$ è mantenuto in memoria (volatile) e viene ricostruito esclusivamente a partire dalle risposte del backend.

5.3.1 Modello Dati Utente

Definizione (U_{view}): Sia

$$U_{view} = (id_u, N_u, t_{last})$$

una tripla che rappresenta un utente in vista UI, dove $id_u \in \mathbb{N}$ è l'identificativo univoco restituito dal backend, N_u è il nome visualizzato, e

$$t_{last} \in \mathbb{N}, \quad [t_{last}] = s,$$

è il timestamp Unix dell'ultimo aggiornamento valido.

Definizione ($\mathbb{S}$): Sia

$$\mathbb{S}$$

l'insieme delle stringhe UTF-8 con lunghezza massima 64 caratteri.

Definizione ($\mathcal{A}$): Sia

$$\mathcal{A} \subset \mathbb{N} \times \mathbb{S} \times \mathbb{N}$$

l'insieme dinamico degli utenti attualmente considerati vicini.

Vincolo (unicità per id_u):

$$\forall (id_u, N, t), (id_u, N', t') \in \mathcal{A} \Rightarrow (N, t) = (N', t').$$

5.3.2 Logica di Aggiornamento

Definizione ($\mathcal{M}$): Sia

$$\mathcal{M} = \mathbb{N} \times \mathbb{S}$$

l'insieme dei match possibili restituiti dal backend.

Definizione (m): Sia

$$m = (id_u, N_u) \in \mathcal{M}$$

un match (istanza) restituito dal backend.

Procedura (onMatchFound): Sia

$$\text{onMatchFound} : \mathcal{M} \rightarrow \emptyset$$

la procedura invocata al tempo corrente t_{unix} .

Algoritmo:

1. Si cerca $U_{view} \in \mathcal{A}$ tale che $U_{view}.id_u = m.id_u$.
2. **Se esiste** $U_{view} = (id_u, \tilde{N}, \tilde{t})$: si aggiorna $\mathcal{A}$ sostituendo tale elemento con

$$(id_u, m.N_u, \max(\tilde{t}, t_{unix})).$$

3. **Se non esiste**: si inserisce in $\mathcal{A}$ l'elemento

$$(M.id_u, m.N_u, t_{unix}).$$

4. Se $\mathcal{A}$ è cambiato, si notifica la UI.

5.3.3 Logica di Pruning

Definizione (τ_{vis}): Sia

$$\tau_{\text{vis}} = \frac{\Delta t_{\text{slot}}}{3}$$

il tempo massimo di permanenza senza refresh. *Uso*: scelta tale che $\tau < \tau_{\text{vis}}$, evitando che la deduplica di ingestione riduca la reattività della vista.

Procedura (**pruneActiveSet**): Sia

$$\text{pruneActiveSet} : \emptyset \rightarrow \emptyset$$

la procedura invocata periodicamente al tempo corrente t_{unix} .

Algoritmo:

1. Si identifica il sottoinsieme scaduto:

$$\mathcal{A}_{\text{exp}} = \{ (id_u, N_u, t_{\text{last}}) \in \mathcal{A} \mid (t_{\text{unix}} - t_{\text{last}}) \geq \tau_{\text{vis}} \}.$$

2. Si rimuove il sottoinsieme scaduto:

$$\mathcal{A} \leftarrow \mathcal{A} \setminus \mathcal{A}_{\text{exp}}.$$

3. Se $\mathcal{A}_{\text{exp}} \neq \emptyset$, si notifica la UI.

6 Modulo iOS (:iosApp)

Il modulo iOS implementa la logica BLE nativa tramite *CoreBluetooth*, rispettando i vincoli di esecuzione in background e la peculiarità iOS per cui il campo *Manufacturer Specific Data* (MSD) non è affidabile in background. Il controllo del flusso resta nativo: il modulo invoca unicamente le procedure pubbliche del modulo condiviso (*Core*) e preserva la semantica FIFO persistente della coda $\mathcal{T}_{\text{queue}}$ e del batch $\mathcal{T}_{\text{batch}}$ (definiti nello *Strato Condiviso (KMM)*).

6.1 BleController

BleController è un singleton che possiede e coordina:

- un CBCentralManager per scanning e connessioni GATT (ruolo *central*);
- un CBPeripheralManager per advertising e server GATT (ruolo *peripheral*);
- lo scheduling delle invocazioni periodiche/event-driven verso KMM:
Core.ensureAdvertisingBatch, Core.getCurrentBid, Core.ingestPacket,
Core.syncQueue.

Tutte le invocazioni verso KMM e tutte le transizioni operative BLE sono serializzate su un'unica coda di esecuzione (single-queue), così che, a parità di stessi eventi nativi e stesso stato persistente, gli effetti osservabili siano deterministici.

6.1.1 Stato foreground/background e commutazione deterministica

La commutazione FG/BG è guidata dallo stato dell'app iOS esposto dal lifecycle (UIApplication/UIScene). BleController non deduce FG/BG dallo stack BLE: aggiorna uno stato interno consultando e ricevendo i callback del sistema operativo.

Definizione (σ): Sia

$$\sigma \in \{\text{FG}, \text{BG}\}$$

lo stato corrente dell'applicazione iOS. *Uso*: determina la modalità di advertising locale e stabilisce se il payload applicativo debba transitare in MSD (FG) oppure via GATT (BG).

Regola (mappatura lifecycle $\rightarrow \sigma$): $\sigma = \text{FG}$ se e solo se l'app è in stato *foreground attivo* (es. `applicationState == active` oppure `scene.activationState == foregroundActive`). In tutti gli altri casi (inattivo inclusa), si impone $\sigma = \text{BG}$ per sicurezza operativa (evita di assumere MSD affidabile durante transizioni, lock screen o interruzioni).

Trigger (transizioni): `BleController` aggiorna σ e applica la riconfigurazione al verificarsi di:

- ingresso in background (`DidEnterBackground/sceneDidEnterBackground`);
- rientro in foreground (`willEnterForeground/sceneWillEnterForeground` e successivo `didBecomeActive/sceneDidBecomeActive`);
- transizione a stato non attivo (`willResignActive/sceneWillResignActive`), che viene trattata come BG.

6.1.2 Implementazione della matrice BLE

La matrice di comportamento è implementata come scelta deterministica tra GAP e GATT in base a σ (per trasmissione) e alla presenza dell'MSD (per ricezione). Le costanti $U_{\text{srv}}, U_{\text{char}}, C_{\text{id}}$ sono definite nella sezione *Matrice di comportamento BLE* di questo documento.

Definizione (msdPresent): Sia

$$\text{msdPresent} \in \{\text{Vero}, \text{Falso}\}$$

il predicato che vale Vero se e solo se il report di discovery contiene un blocco MSD tale che (i) i primi 2 B sono B1 00 (little-endian di $C_{\text{id}} = 0x00B1$) e (ii) la lunghezza totale del dato applicativo è 18 B (header $C_{\text{id}} + 16$ B). *Uso:* seleziona la lettura diretta (GAP) oppure il fallback connesso (GATT) in scanning.

Regola (Advertising iOS):

- se $\sigma = \text{FG}$, l'advertising è GAP: in advertisement sono presenti U_{srv} e l'MSD che porta B_{id} ;
- se $\sigma = \text{BG}$, l'advertising è GATT: in advertisement è presente U_{srv} (discovery e filtro), mentre B_{id} è leggibile via caratteristica U_{char} del servizio pubblicato.

Regola (Scanning iOS): lo scanning è sempre filtrato su U_{srv} . Per ogni peripheral scoperto:

- se $\text{msdPresent} = \text{Vero}$, si estrae B_{id} in GAP e si invoca `Core.ingestPacket`;
- se $\text{msdPresent} = \text{Falso}$, si attiva il fallback GATT e si legge U_{char} .

Questa regola realizza la matrice in modo completo: in particolare copre il caso “trasmittente iOS in BG” dove l'MSD è assente/non affidabile, quindi $\text{msdPresent} = \text{Falso} \Rightarrow \text{GATT}$.

6.1.3 Precondizioni di piattaforma

L'operatività è subordinata allo stato `CoreBluetooth` e ai permessi.

- Se lo stato Bluetooth non è utilizzabile (`poweredOff/unauthorized/unsupported/resetting`), `BleController` forza `stopAdvertising`, interrompe le sessioni GATT in corso e sospende lo scanning.
- Le modalità in background richiedono i *Background Modes* per Bluetooth (`bluetooth-central` e `bluetooth-peripheral`) e le *usage description* previste dal sistema; in assenza, gli invarianti di scheduling restano validi ma l'esecuzione in background non è garantita.

Queste condizioni non modificano mai $\mathcal{T}_{\text{batch}}$ o $\mathcal{T}_{\text{queue}}$ direttamente: ogni modifica persistente avviene solo tramite KMM.

6.1.4 Inizializzazione e *State Restoration*

BleController inizializza CBCentralManager e CBPeripheralManager con *restoration identifiers* distinti e abilita la *state restoration*. Su ripristino, ricostruisce le invarianti operative in modo idempotente:

- riattiva lo scanning filtrato su U_{srv} se lo stato Bluetooth è utilizzabile;
- ricostruisce (se necessario) il server GATT (servizio U_{srv} + caratteristica U_{char});
- riallinea l'advertising alla regola imposta da σ (sezione successiva);
- riattiva i timer (sezione *Scheduling*).

La *restoration* non implica alcun enqueue/dequeue: la FIFO $\mathcal{T}_{\text{queue}}$ e il batch $\mathcal{T}_{\text{batch}}$ sono già persistenti e restano la sorgente di verità per KMM.

6.1.5 Advertising e reconfigurazione FG/BG

BleController non costruisce B_{id} : richiede al KMM il payload di slot corrente tramite `Core.getCurrentBid` (definita nello *Strato Condiviso (KMM)*). Il processo di advertising è deterministico e dipende da σ .

Composizione MSD (solo $\sigma = \text{FG}$): L'MSD trasporta un vettore di 18 B definito come

$$D = C_{\text{id}} \parallel B_{\text{id}},$$

dove C_{id} occupa i primi 2 B e $B_{\text{id}} \in \{0,1\}^{128}$ i successivi 16 B. In scanning la validazione è: $|D| = 18 \text{ B}$ e $D[0..1] = C_{\text{id}}$ (endianness conforme all'API nativa).

Procedura (`applyAdvertisingMode`): La procedura viene invocata quando:

- lo stato Bluetooth diventa utilizzabile (`poweredOn`);
- σ cambia ($\text{FG} \leftrightarrow \text{BG}$);
- cambia lo slot temporale S_{slot} (sezione successiva).

I passi sono:

1. invocare `Core.ensureAdvertisingBatch()` (opportunistico: se la copertura è sufficiente non produce modifiche persistenti; definita nello *Strato Condiviso (KMM)*);
2. invocare `Core.getCurrentBid()`. Se restituisce `null`, eseguire `stopAdvertising` e non esporre dato applicativo (né in MSD né via U_{char}) finché un batch valido non viene ripristinato;
3. se `Core.getCurrentBid()` restituisce un valore, allora:
 - se $\sigma = \text{FG}$: pubblicare l'advertising in GAP includendo U_{srv} e MSD D ;
 - se $\sigma = \text{BG}$: pubblicare l'advertising in GATT includendo U_{srv} e assicurare l'esposizione del server GATT con caratteristica U_{char} leggibile.

Ogni cambio modalità forza una riconfigurazione esplicita (`stopAdvertising` seguito da `startAdvertising`) così che l'advertisement attivo sia sempre coerente con σ .

6.1.6 Timer di slot

Per garantire che l'advertising in FG contenga sempre il B_{id} dello slot corrente, BleController riallinea al cambio di slot S_{slot} (definito nello *Strato Condiviso (KMM)* e in *Architettura BLE lato Client e Server*).

Scheduling: Dato il tempo corrente t_{unix} e Δt_{slot} (definiti nello *Strato Condiviso (KMM)*), `BleController` calcola localmente l'istante del prossimo bordo di slot e programma un timer a singola scadenza. Alla scadenza:

- invoca `applyAdvertisingMode` (che a sua volta richiama `Core.getCurrentBid`);
- riprogramma il timer sul bordo successivo.

Questo elimina drift di scheduling: l'aggiornamento è sempre ancorato ai bordi di slot, non a un periodo accumulativo.

6.1.7 Server GATT

Quando $\sigma = \text{BG}$, il payload applicativo è esposto via GATT utilizzando U_{srv} e U_{char} (definiti nella sezione *Matrice di comportamento BLE*).

Pubblicazione: `BleController` assicura che:

- esista un servizio pubblicato con UUID U_{srv} ;
- esista una caratteristica leggibile con UUID U_{char} .

Semantica lettura caratteristica: Alla richiesta di lettura su U_{char} , `BleController` invoca `Core.getCurrentBid()`:

- se il risultato è definito, risponde con i 16 B del B_{id} corrente;
- se il risultato è `null`, risponde con errore GATT (dato non disponibile) e mantiene l'advertising sospeso finché `Core.ensureAdvertisingBatch()` non ripristina copertura e `Core.getCurrentBid()` torna definita.

In $\sigma = \text{BG}$ non è necessario riavviare l'advertising a ogni slot ai fini della correttezza del dato remoto: il valore letto è determinato al momento della read dalla `getCurrentBid`. Il timer di slot resta comunque attivo per gestire la riconfigurazione globale e i casi $\sigma = \text{FG}$.

6.1.8 Scanning, acquisizione GAP e ingestione KMM

Lo scanning è sempre filtrato su U_{srv} . Per ogni evento di discovery, `BleController` applica:

1. se `msdPresent = Vero`, estrae

$$B_{\text{id}} = D[2..17] \in \{0, 1\}^{128}$$

e invoca immediatamente `Core.ingestPacket( $B_{\text{id}}$ )`;

2. se `msdPresent = Falso`, attiva il fallback GATT (sezione successiva).

La deduplica e l'eventuale enqueue persistente sono interamente responsabilità di KMM tramite `Core.ingestPacket` (che applica la cache volatile e delega l'I/O a `SecureRepository.enqueue`).

6.1.9 Fallback GATT e controllo dei tentativi

Quando `msdPresent = Falso`, `BleController` tenta la lettura connessa di U_{char} . Per evitare tempeste di connessione e connessioni duplicate concorrenti verso lo stesso peripheral, `BleController` mantiene un throttle volatile per identificatore di peripheral (implementativamente: chiave opaca fornita da `CoreBluetooth` e stabile nel ciclo di vita del processo).

Definizione ($\mathcal{P}$): Sia

$$\mathcal{P} \subset \{(\iota, t) \mid t \in \mathbb{N}\}$$

una cache volatile che associa a ciascun peripheral ι l'ultimo timestamp t di avvio di un tentativo GATT. *Uso*: limita la frequenza di tentativi GATT per peripheral con finestra minima τ (definita nello *Strato Condiviso (KMM)*), impedendo connessioni ripetute non produttive.

Regola (throttle): Un tentativo GATT verso ι è ammesso se e solo se

$$\neg \exists (\iota, t) \in \mathcal{P} \mid (t_{\text{unix}} - t) < \tau.$$

All'avvio del tentativo si aggiorna $\mathcal{P}$ sostituendo la coppia per ι con (ι, t_{unix}) .

Procedura (GATT read): Per un peripheral ι ammesso:

1. connect (connessione GATT);
2. discovery del servizio U_{srv} ;
3. discovery della caratteristica U_{char} ;
4. readValue su U_{char} ;
5. se la lettura restituisce un valore di 16 B, interpretato come $B_{\text{id}} \in \{0, 1\}^{128}$, invocare `Core.ingestPacket( $B_{\text{id}}$ )`;
6. cancelPeripheralConnection e cleanup della sessione.

Gestione errori: Qualunque errore tecnico (connessione, discovery, read) termina la procedura senza invocare `Core.ingestPacket`. Il peripheral resta soggetto al throttle di $\mathcal{P}$. I timeout sono implementati come cancellation esplicita della connessione se la sequenza non completa entro una finestra finita coerente con la logica di throttle (tipicamente non superiore alla finestra minima tra tentativi).

6.1.10 Scheduling invocazioni KMM e garanzie

Oltre al timer di bordo slot, `BleController` programma un timer periodico *best-effort* (eseguito quando il processo ha tempo di esecuzione) con periodo τ (definito nello *Strato Condiviso (KMM)*). A ogni tick invoca in ordine:

1. `Core.ensureAdvertisingBatch()`;
2. `Core.syncQueue()`.

Non è richiesto chiamare `ResolvedRepository.pruneActiveSet` dal layer iOS: la potatura è già garantita (i) internamente a `Core.syncQueue` (definizione nello *Strato Condiviso (KMM)*) quando la risoluzione viene eseguita e (ii) dalla UI/driver nativi tramite tick periodici se si desidera pruning anche in assenza totale di rete; in tal caso l'invocazione è lecita perché agisce unicamente su $\mathcal{A}$ (volatile).

Garanzie:

- `Core.ensureAdvertisingBatch` è idempotente: in assenza di rete o su errore non modifica $\mathcal{T}_{\text{batch}}$.
- `Core.syncQueue` rispetta la semantica 1:1: ogni tick può processare al più un elemento (la testa FIFO) e, su errore di rete/risposta non valida, $\mathcal{T}_{\text{queue}}$ resta invariata.
- `Core.ingestPacket` è l'unica via per inserire in FIFO: garantisce deduplica volatile e persistenza transazionale tramite `SecureRepository.enqueue`.

6.1.11 Trigger evento-driven

Oltre ai timer, `BleController` invoca KMM in punti deterministici:

- su `poweredOn`: invoca `applyAdvertisingMode` e avvia/scandisce su U_{srv} ;
- su cambio σ ($\text{FG} \leftrightarrow \text{BG}$): invoca `applyAdvertisingMode` (commuta $\text{GAP} \leftrightarrow \text{GATT}$) senza alterare l'invariante di scanning filtrato;
- su acquisizione di un payload (GAP o GATT): invoca immediatamente `Core.ingestPacket`;
- su ripristino stato (state restoration): riapplica `applyAdvertisingMode`, riattiva scan e timers, senza eseguire side-effect persistenti diretto (solo via KMM).

La combinazione di: (i) riconfigurazione evento-driven per σ e per disponibilità BLE, (ii) timer di bordo slot, (iii) tick periodico τ per batch e coda, è sufficiente a garantire che (a) l'advertising sia sempre coerente con σ e con lo slot corrente, (b) ogni B_{id} acquisito sia consegnato a KMM in modo idempotente, (c) la risoluzione verso backend avanzi in FIFO preservando la semantica 1:1.

6.2 Flussi di Esecuzione

Di seguito sono riportati i diagrammi di flusso che illustrano le principali logiche decisionali e operative del modulo iOS, in conformità con le definizioni testuali precedenti.

6.2.1 Ciclo di Vita e Gestione Stato

Questo diagramma mostra come `BleController` gestisce l'inizializzazione, la *State Restoration* e le transizioni di stato dell'applicazione (Foreground/Background), aggiornando di conseguenza la modalità operativa σ .

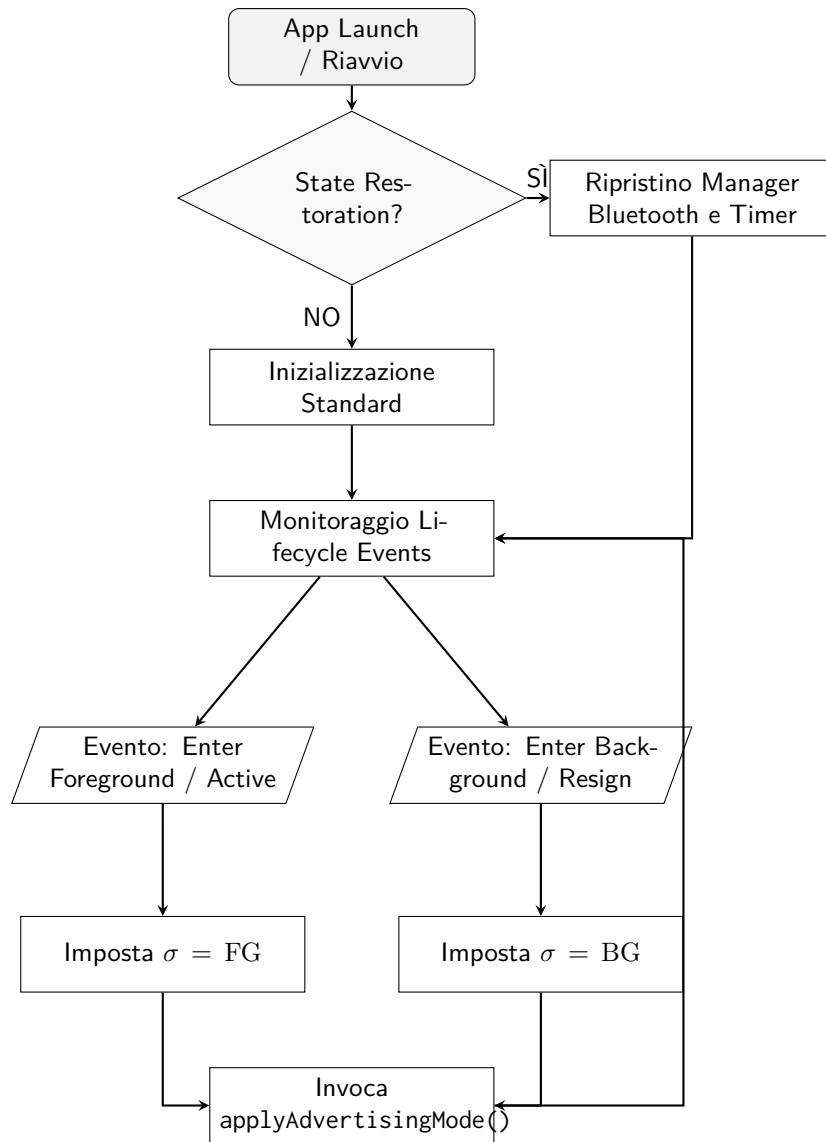


Figure 2: Flusso del Ciclo di Vita e aggiornamento di σ

6.2.2 Logica di Advertising

Questo diagramma dettaglia la procedura `applyAdvertisingMode`, invocata su cambi di stato, timer di slot o eventi Bluetooth. Mostra la decisione deterministica tra advertising GAP e GATT basata su σ e la dipendenza dal KMM.

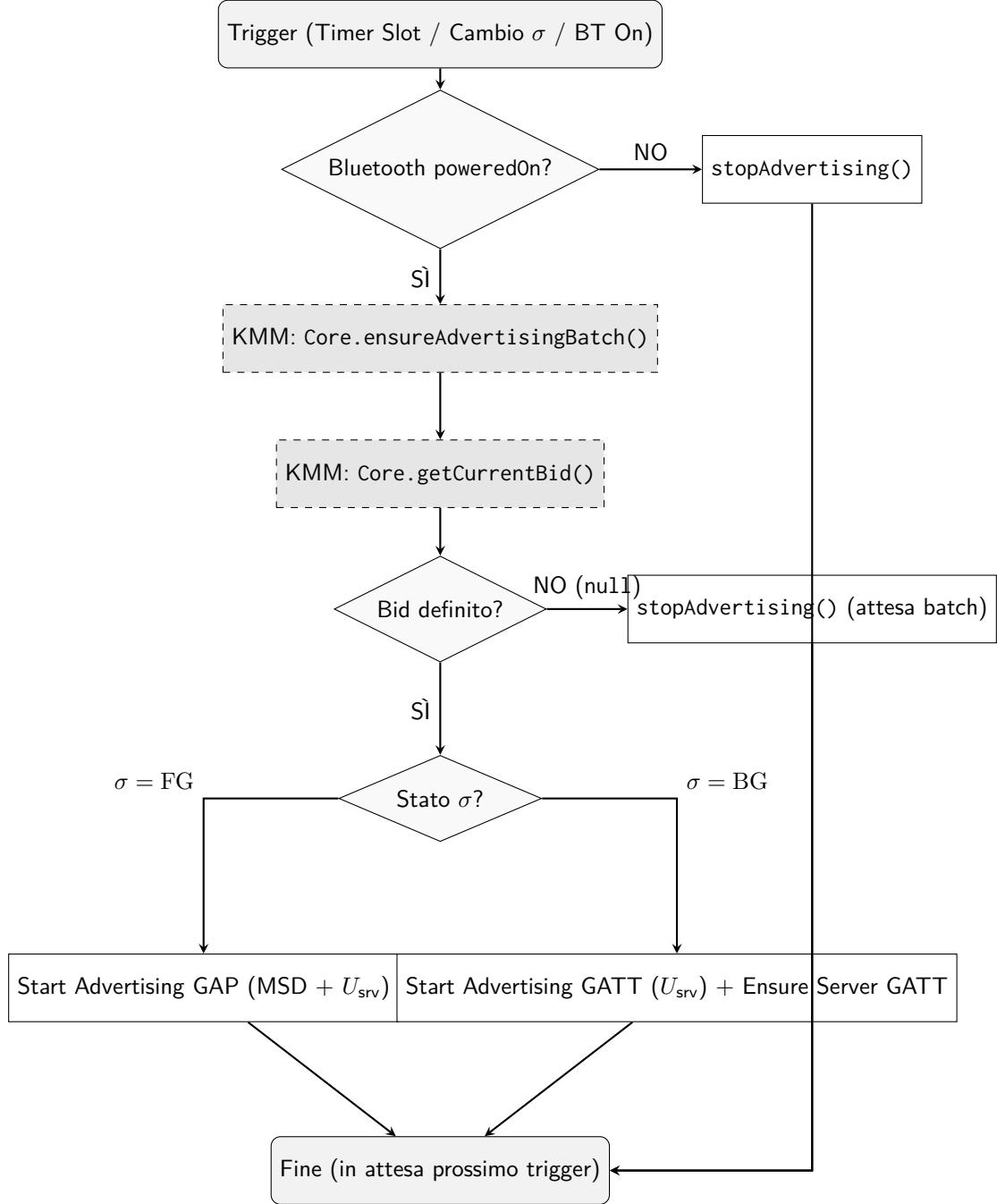


Figure 3: Flusso decisionale dell'Advertising iOS

6.2.3 Scanning, Fallback GATT e Ingestione

Questo diagramma illustra la logica ibrida di ricezione. Mostra la gestione dell'evento di discovery, il controllo della presenza MSD, l'eventuale fallback con connessione GATT (soggetto a throttle $\mathcal{P}$) e l'ingestione finale del payload verso il KMM.

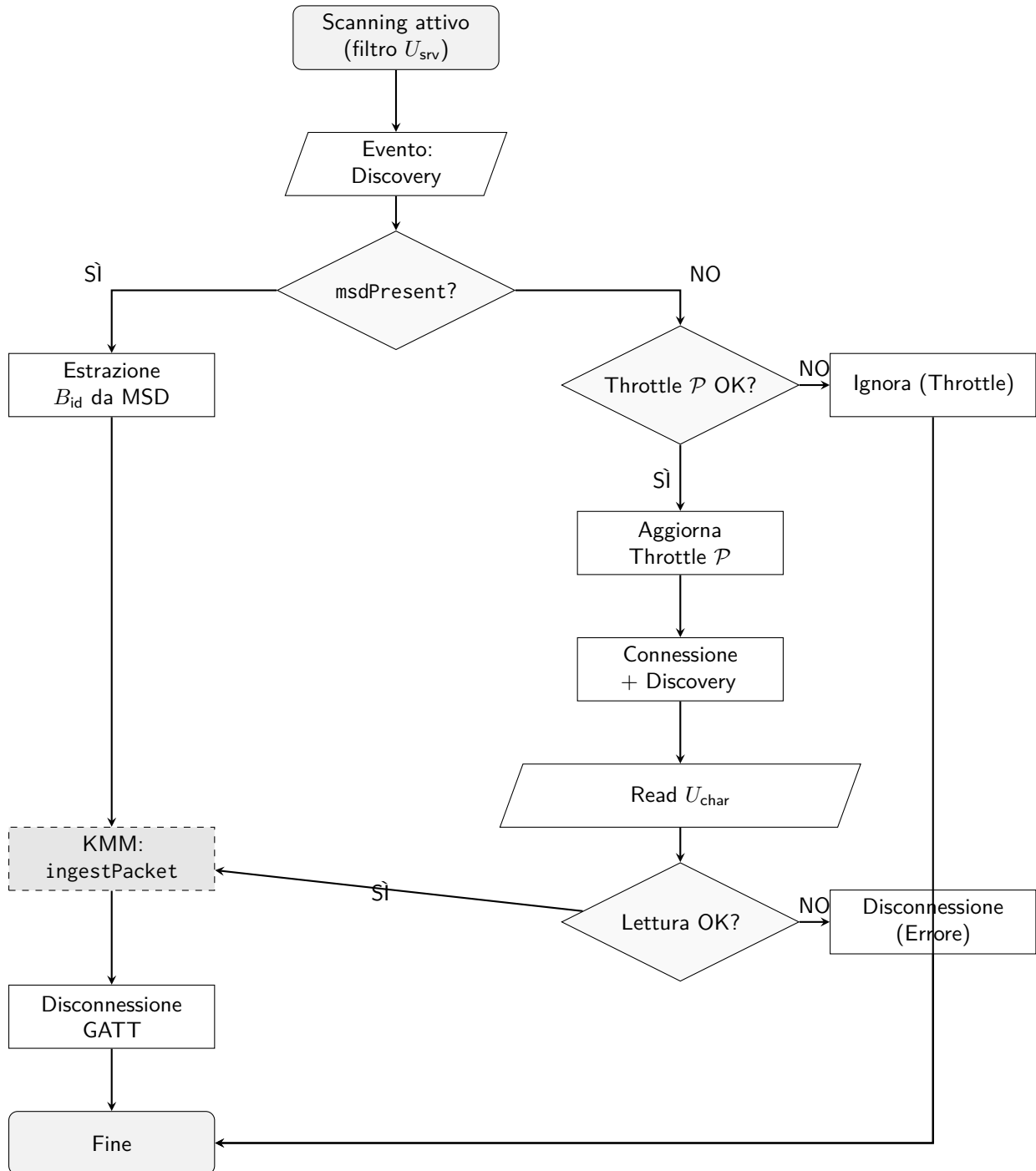


Figure 4: Flusso di Scanning ibrido e Ingestione

7 Modulo Android (:androidApp)

Il modulo Android implementa la logica BLE nativa tramite lo stack `android.bluetooth.*`. Il controllo del flusso resta nativo: l'esecuzione è mantenuta continua tramite `BleService`

(*Foreground Service*) e tutte le invocazioni al modulo condiviso avvengono *solo* tramite le procedure pubbliche di Core (KMM), preservando la semantica FIFO della coda persistente $\mathcal{T}_{\text{queue}}$ e del batch $\mathcal{T}_{\text{batch}}$ (definiti nello *Strato Condiviso (KMM)*).

7.1 BleService

BleService è un *Foreground Service* che possiede e coordina:

- BluetoothLeAdvertiser per advertising (ruolo *peripheral*);
- BluetoothLeScanner per scanning (ruolo *central*);
- BluetoothGattServer per server GATT quando la modalità locale richiede GATT;
- lo scheduling (timer) delle invocazioni periodiche/event-driven verso KMM: `Core.ensureAdvertisingBatch`, `Core.getCurrentBid`, `Core.ingestPacket`, `Core.syncQueue`.

Ogni transizione BLE e ogni invocazione KMM è serializzata su un'unica coda di esecuzione (single-queue), prevenendo interleaving non deterministici.

7.1.1 Stato foreground/background e politica in background

Si riusa $\sigma \in \{\text{FG}, \text{BG}\}$ come stato dell'applicazione (foreground/background), già introdotto nel documento. In Android, σ è derivato dal lifecycle di processo (es. *process in foreground* se esiste almeno una Activity in stato RESUMED; altrimenti $\sigma = \text{BG}$). *Uso*: abilita la commutazione deterministica richiesta dalla matrice nel solo caso Android $\text{BG} \Rightarrow \text{GAP/GATT}$.

Definizione (ρ): Sia

$$\rho \in \{\text{GAP}, \text{GATT}\}$$

la politica configurabile di trasporto locale quando $\sigma = \text{BG}$. *Uso*: realizza la cella *BG Android* = *GAP/GATT*: se $\rho = \text{GAP}$ si continua a trasmettere B_{id} in MSD anche in background; se $\rho = \text{GATT}$ si trasmette per discovery solo U_{srv} e si rende B_{id} leggibile via U_{char} . *Origine (vincolante)*: ρ è scelto dall'utente dalla UI tramite un controllo a scelta esclusiva con due soli valori: GAP e GATT. *Persistenza (vincolante)*: su Android il valore è salvato in EncryptedSharedPreferences con chiave `bg_transport_mode` e letto da BleService a ogni invocazione di `applyAdvertisingMode`. *Default (vincolante)*: al primo avvio, se la chiave non esiste, vale $\rho = \text{GAP}$.

7.1.2 Implementazione della matrice BLE

Le costanti $U_{\text{srv}}, U_{\text{char}}, C_{\text{id}}$ sono definite nella sezione *Matrice di comportamento BLE*.

Si riusa il predicato `msdPresent` (già introdotto nel documento) come discriminante tra lettura diretta in GAP e fallback GATT in scanning.

Regola (Advertising Android):

- se $\sigma = \text{FG}$, l'advertising è GAP: il payload B_{id} è inserito in MSD;
- se $\sigma = \text{BG}$, allora:
 - se $\rho = \text{GAP}$, l'advertising resta GAP (MSD presente);
 - se $\rho = \text{GATT}$, l'advertising è GATT: l'advertisement contiene U_{srv} e B_{id} è leggibile via U_{char} .

Regola (Scanning Android): lo scanning è sempre filtrato su U_{srv} . Per ogni peripheral scoperto:

- se `msdPresent` = Vero, si acquisisce B_{id} in GAP e si invoca `Core.ingestPacket`;
- se `msdPresent` = Falso, si attiva fallback GATT e si legge U_{char} tramite `GattClientManager`.

7.1.3 Precondizioni di piattaforma

BleService applica una precondizione necessaria prima di attivare advertising/scanning:

- adapter Bluetooth disponibile e abilitato;
- supporto BLE e supporto advertising (device-dependent);
- permessi runtime coerenti con la versione Android:
 - Android ≥ 12 : BLUETOOTH_SCAN, BLUETOOTH_ADVERTISE, BLUETOOTH_CONNECT;
 - Android < 12 : permessi BLE + vincolo storico di location per scan (es. ACCESS_FINE_LOCATION).

Se una precondizione non è soddisfatta, BleService impone `stopAdvertising` e `stopScan` e chiude eventuali sessioni GATT pendenti. Nessuna modifica su $\mathcal{T}_{\text{batch}}$ o $\mathcal{T}_{\text{queue}}$ avviene fuori da KMM.

7.1.4 Inizializzazione e ripristino invarianti operative

All'avvio del servizio (o quando lo stato Bluetooth torna utilizzabile), BleService ricostruisce in modo idempotente le invarianti:

- avvia scanning filtrato su U_{srv} ;
- riallinea l'advertising alla regola imposta da (σ, ρ) tramite `applyAdvertisingMode` (definita sotto);
- attiva:
 - timer di bordo slot (riallineamento a S_{slot});
 - timer periodico di periodo τ per batch/coda.

Le operazioni sono idempotenti: ripetizioni non producono effetti persistenti aggiuntivi, perché ogni side-effect su disco è mediato da KMM.

7.1.5 Advertising e commutazione

BleService non costruisce B_{id} : richiede il payload a KMM tramite `Core.getCurrentBid` (definita nello *Strato Condiviso (KMM)*).

Composizione MSD (solo modalità GAP): In GAP si usa la struttura MSD coerente con il documento:

$$D = C_{\text{id}} \parallel B_{\text{id}}.$$

In Android, l'API separa l'header C_{id} dal payload: si configura il Company ID a valore C_{id} e si inseriscono i soli 16 B di B_{id} come dati; la validazione `msdPresent` resta definita in modo equivalente perché l'*AD structure* risultante contiene esattamente header + payload.

Procedura (`applyAdvertisingMode`): La procedura è invocata quando:

- le precondizioni BLE diventano vere (Bluetooth ON + permessi);
- σ cambia (FG $\leftrightarrow$ BG);
- ρ cambia (solo se $\sigma = \text{BG}$);
- cambia lo slot temporale S_{slot} (timer di slot).

I passi sono:

1. invocare `Core.ensureAdvertisingBatch()` per garantire copertura del batch in condizioni di rete;
2. richiedere $b \leftarrow \text{Core.getCurrentBid}()$;

3. se $b = \text{null}$, eseguire:

stopAdvertising

e, se un server GATT era attivo per trasporto locale GATT, lasciarlo pubblicato ma rispondere alle read con errore (sezione *Server GATT*);

4. se $b \neq \text{null}$, determinare la modalità locale:

$$\mu = \begin{cases} \text{GAP} & \text{se } \sigma = \text{FG} \\ \rho & \text{se } \sigma = \text{BG} \end{cases}$$

e applicare:

- se $\mu = \text{GAP}$: **stopAdvertising** $\rightarrow$ **startAdvertising** includendo U_{srv} e MSD con b ;
- se $\mu = \text{GATT}$: **stopAdvertising** $\rightarrow$ **ensureGattServerPublished** (servizio U_{srv} + caratteristica U_{char}) $\rightarrow$ **startAdvertising** includendo U_{srv} (nessun MSD).

La procedura è idempotente: invocazioni ripetute producono uno stato BLE equivalente, a parità di (σ, ρ) e di b .

7.1.6 Timer di slot

Si riusano $t_{\text{unix}}, \Delta t_{\text{slot}}, S_{\text{slot}}$ definiti nello *Strato Condiviso (KMM)*.

Dato t_{unix} , **BleService** programma un timer a singola scadenza sul bordo successivo:

$$t_{\text{next}} = (S_{\text{slot}} + 1) \cdot \Delta t_{\text{slot}}.$$

Alla scadenza, invoca **applyAdvertisingMode** e riprogramma sul bordo successivo. Questo garantisce assenza di drift: l'aggiornamento è ancorato ai bordi di slot, non a un periodo accumulativo.

7.1.7 Server GATT

Quando la modalità locale è GATT, **BleService** espone un server GATT che permette la lettura di B_{id} tramite U_{char} .

Pubblicazione:

- servizio con UUID U_{srv} ;
- caratteristica leggibile con UUID U_{char} .

Semantica di read su U_{char} : Alla richiesta di lettura, **BleService** risponde con:

$$b \leftarrow \text{Core.getCurrentBid}().$$

Se $b \neq \text{null}$, restituisce i 16 B del payload; se $b = \text{null}$, risponde con errore GATT (dato non disponibile). La correttezza temporale è garantita perché b è valutato al tempo della read (coerente con S_{slot} corrente).

7.1.8 Scanning e ingestione GAP

Lo scanning è sempre filtrato per U_{srv} . Alla discovery di un peripheral:

1. se $\text{msdPresent} = \text{Vero}$, si estrae $B_{\text{id}} \in \{0, 1\}^{128}$ e si invoca immediatamente

Core.ingestPacket(B_{id});

2. se `msdPresent = Falso`, si attiva fallback GATT (sezione `GattClientManager`).

La deduplica volatile e l'eventuale enqueue persistente sono interamente responsabilità di `Core.ingestPacket` (KMM).

7.1.9 Scheduling invocazioni KMM

Oltre al timer di slot, `BleService` programma un timer periodico *best-effort* di periodo τ (definito nello *Strato Condiviso* (KMM)). A ogni tick invoca in ordine:

1. `Core.ensureAdvertisingBatch()`;
2. `Core.syncQueue()`.

Queste invocazioni preservano gli invarianti:

- se `ensureAdvertisingBatch` fallisce (rete/errore), $\mathcal{T}_{\text{batch}}$ resta invariata;
- se `syncQueue` fallisce (rete/risposta non valida), $\mathcal{T}_{\text{queue}}$ resta invariata e la FIFO è preservata;
- la semantica 1:1 è garantita da `Core.syncQueue` (al più un elemento per tick).

7.1.10 Trigger evento-driven

Oltre ai timer, `BleService` invoca KMM in punti deterministici:

- su soddisfazione precondizioni BLE (BT ON + permessi): `applyAdvertisingMode` e avvio scanning;
- su cambio σ o ρ : `applyAdvertisingMode` (commuta deterministica GAP↔GATT quando richiesto);
- su acquisizione payload (GAP o GATT): invocazione immediata `Core.ingestPacket`;
- su errore di advertising/scanning (callback di failure o stop imposto dal sistema): riapplicazione idempotente del pipeline via `applyAdvertisingMode` e restart scanning quando le precondizioni tornano valide.

7.2 GattClientManager

`GattClientManager` implementa la sequenza di connessione e read necessaria quando lo scanner rileva U_{srv} ma `msdPresent = Falso` (caso tipico: trasmittente iOS in BG, oppure Android in BG con $\rho = \text{GATT}$).

Si riusano ι (identificatore opaco di peripheral) e $\mathcal{P}$ (cache volatile per throttle dei tentativi GATT) già introdotti nel documento, così come τ (finestra minima tra tentativi).

7.2.1 Regola

Un tentativo GATT verso ι è ammesso se e solo se:

$$\neg \exists (\iota, t) \in \mathcal{P} \mid (t_{\text{unix}} - t) < \tau.$$

All'avvio del tentativo, $\mathcal{P}$ viene aggiornata sostituendo l'eventuale coppia per ι con (ι, t_{unix}) . Tale aggiornamento, eseguito all'inizio, impedisce connessioni duplicate concorrenti verso lo stesso ι (ogni nuovo tentativo cadrebbe nella finestra τ).

7.2.2 Procedura

Dato un peripheral ι eleggibile:

1. connessione GATT (`connectGatt` con `autoConnect` disabilitato);

2. discovery servizi, selezione del servizio U_{srv} ;
3. discovery/lookup della caratteristica U_{char} ;
4. read `readCharacteristic` su U_{char} ;
5. se la lettura produce un valore $B_{\text{id}} \in \{0, 1\}^{128}$, invocazione

`Core.ingestPacket( $B_{\text{id}}$ );`

6. disconnessione e rilascio risorse (`disconnect` e `close`) per evitare leak di sessioni.

7.2.3 Gestione errori e timeout

Qualunque fallimento tecnico (connessione, discovery, read) termina la procedura senza invocare `Core.ingestPacket`. La sessione viene chiusa e il peripheral resta soggetto alla regola di throttle tramite $\mathcal{P}$. Un tentativo che non completa entro una finestra massima coerente con la politica di throttle viene abortito con disconnessione esplicita; la scelta della finestra massima è vincolata a essere finita e non superiore, per ordine di grandezza, alla scala temporale già usata per evitare retry aggressivi (cioè τ).

7.3 Flussi di Esecuzione Android

Di seguito sono riportati i diagrammi di flusso che illustrano le principali logiche decisionali e operative del modulo Android, riflettendo la gestione tramite *Foreground Service* e timer locali.

7.3.1 Ciclo di Vita e Gestione Stato

Questo diagramma mostra come `BleService` gestisce l'avvio, le precondizioni (Bluetooth/Permessi) e la determinazione della modalità operativa locale basata sullo stato del processo (σ) e sulla configurazione utente (ρ).

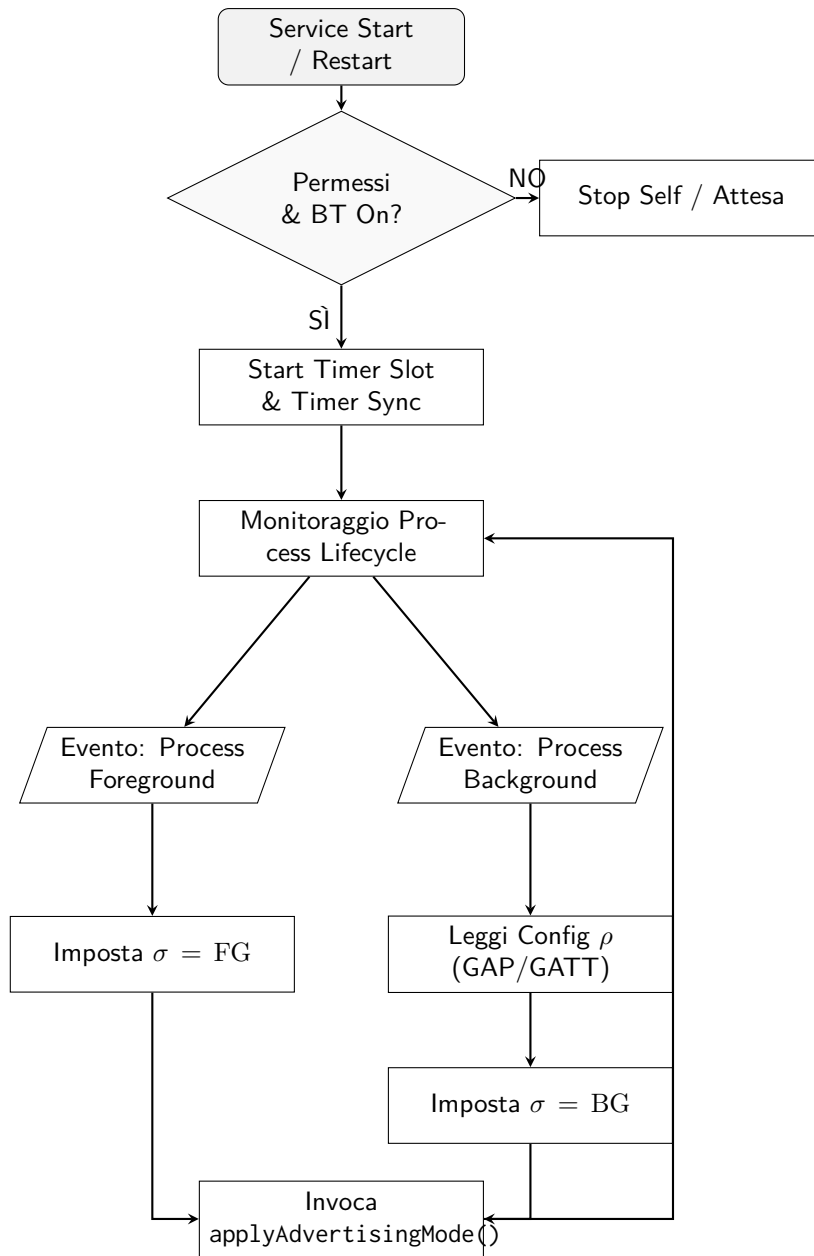


Figure 5: Flusso Android: Ciclo di vita e aggiornamento stato

7.3.2 Logica di Advertising

Questo diagramma dettaglia la procedura `applyAdvertisingMode`. Evidenzia la gestione idempotente e la selezione della modalità di trasporto (μ) in base alla combinazione di σ e ρ , interagendo con il KMM per ottenere il payload.

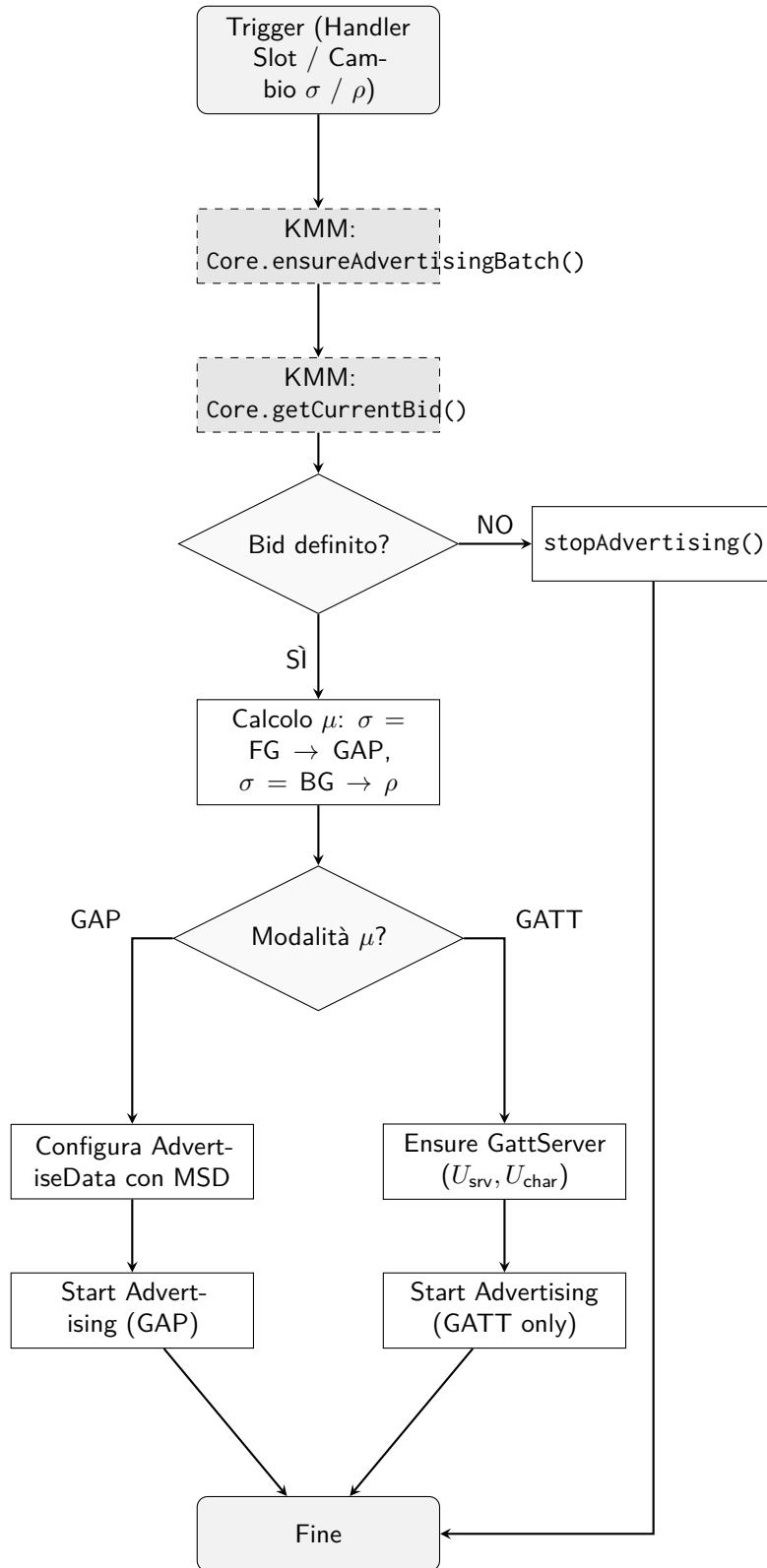


Figure 6: Flusso decisione Advertising Android (μ)

7.3.3 Scanning e Fallback GATT

Questo diagramma illustra la logica di ricezione nel `BleService` e la delega al `GattClientManager`. Mostra il check MSD, il throttle locale per le connessioni e l'uso finale di `Core.ingestPacket`.

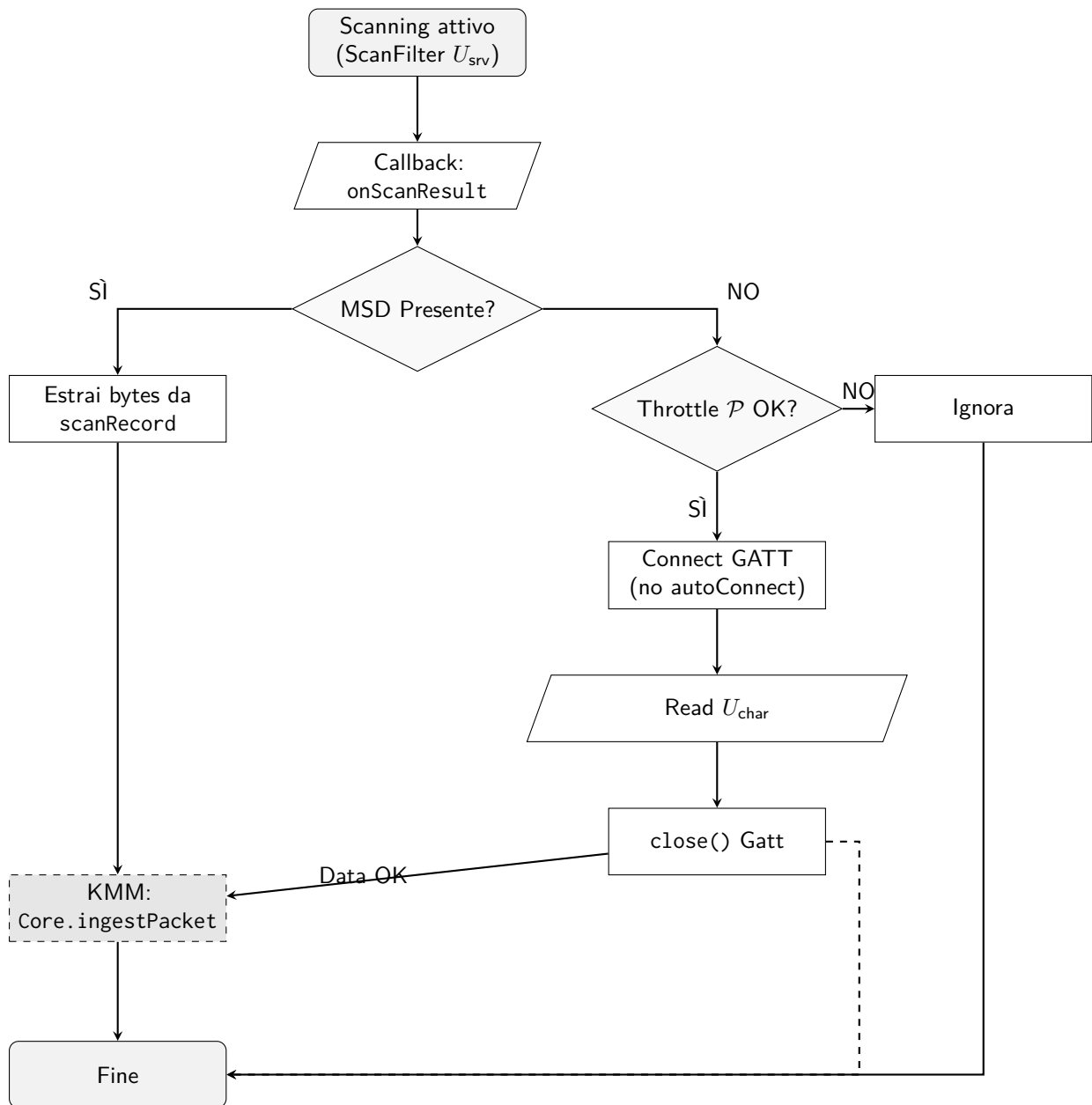


Figure 7: Flusso Android: Scanning GAP e Fallback GATT